

Programmation et Algorithmique I

Veillez respecter scrupuleusement les consignes suivantes :

- répondez à chaque problème **sur des feuilles séparées** et utilisez uniquement les feuilles de réponses qui vous sont données ;
- indiquez votre nom, prénom et section sur chaque feuille de réponses ;
- les notes de cours et personnelles **ne sont pas autorisées** ;
- si ce n'est pas explicitement demandé dans l'énoncé, **vous ne devez pas tester que les préconditions sont respectées**.

Problème 1 : déplacements dans une maison

Considérez une maison comportant un ensemble de pièces. Certaines de ces pièces sont reliées entre elles par une porte, et l'on peut donc passer de l'une à l'autre en empruntant cette porte. On suppose que les différentes pièces de la maison sont stockées dans une liste de chaînes de caractères, par exemple ['hall', 'salon', 'cuisine']. On suppose également que les différentes portes sont stockées dans une liste de couples ('p1', 'p2') indiquant que l'on peut passer de la pièce 'p1' à la pièce 'p2'. Notez bien que les portes ne s'ouvrent pas forcément dans les deux sens. Ainsi, une porte ('p1', 'p2') n'implique pas forcément que l'on peut passer de la pièce 'p2' à la pièce 'p1'.

On vous demande d'implémenter un algorithme qui, étant donnée une pièce de départ, une pièce d'arrivée ainsi qu'une liste des portes de la maison, retourne le chemin (pièce par pièce) permettant de se rendre de la pièce de départ à la pièce d'arrivée. Par exemple, si la maison contient les pièces ['a', 'b', 'c', 'd'] ainsi que les portes [('a', 'b'), ('b', 'c'), ('a', 'c'), ('b', 'a'), ('d', 'b')], et que l'on vous demande de passer de la pièce 'd' à la pièce 'c', l'algorithme peut retourner la liste suivante : ['d', 'b', 'c'] (notez que l'on peut également passer de 'd' à 'c' en suivant le chemin ['d', 'b', 'a', 'c']).

Questions

Implémentez les fonctions récursives suivantes **dans l'ordre** :

- a) Une fonction **récursive** `peut_aller(piece1, piece2, portes, eviter)` qui retourne la valeur booléenne vrai si et seulement s'il est possible, en partant de la `piece1`, de rejoindre la `piece2` en respectant les `portes` (une liste de tuples), sans passer deux fois par la même pièce. Le paramètre `eviter` consiste en une liste des pièces à éviter. Lors du premier appel récursif, cette liste est vide.
- b) Une fonction **récursive** `chemin(piece1, piece2, portes, eviter)` qui retourne un chemin sous la forme d'une liste des pièces à traverser (celles de départ et d'arrivée incluses) pour aller de la `piece1` à la `piece2`. Si aucun chemin n'existe, la fonction retourne `None`. Les paramètres `portes` et `eviter` sont identiques à ceux de la fonction précédente.
- c) **Bonus** : Une fonction **récursive** `plus_court_chemin(piece1, piece2, portes, eviter)` qui retourne **le plus court chemin** sous la forme d'une liste des pièces à traverser (celles de départ et d'arrivée incluses) pour aller de la `piece1` à la `piece2`. Si aucun chemin n'existe, la fonction retourne `None`. Les paramètres `portes` et `eviter` sont identiques à ceux de la fonction précédente.

Problème 2 : Aire d'un polygone

Soit P un polygone convexe à n côtés ($n > 2$) dans le plan $\mathbb{R}^2$. L'objectif de cet exercice est de concevoir un algorithme **récuratif** qui calcule l'aire de P .

Considérons $p_1, \dots, p_n$ les sommets de P énumérés **dans le sens des aiguilles d'une montre** (le choix du premier sommet p_1 n'a pas d'importance). Par exemple, on peut représenter un carré de côté 1 par la liste $[(0,0), (0,1), (1,1), (1,0)]$.

Le cas de base de l'algorithme récuratif est réalisé lorsque le polygone P possède 3 sommets. Dans ce cas, le polygone est un triangle et son aire est calculée comme suit :

- notons $p_1 = (x_1, y_1)$, $p_2 = (x_2, y_2)$ et $p_3 = (x_3, y_3)$;
- l'aire du triangle est égale à

$$\frac{1}{2} \left| \det \begin{pmatrix} x_2 - x_1 & x_3 - x_1 \\ y_2 - y_1 & y_3 - y_1 \end{pmatrix} \right| = \frac{1}{2} |(x_2 - x_1) \cdot (y_3 - y_1) - (x_3 - x_1) \cdot (y_2 - y_1)|$$

Pour le cas récuratif, nous avons que le nombre de sommets du polygone P est supérieur ou égal à 4. L'idée d'une étape récurative est de calculer l'aire de P à partir de deux polygones convexes à n_1 et n_2 côtés, avec $n_1 < n$ et $n_2 < n$, construits comme suit (illustration en figure 1) :

- on choisit deux sommets distincts p_i et p_j ($1 \leq i < j \leq n$) de P **non consécutifs** d'un point de vue géométrique (donc p_1 et p_n sont considérés consécutifs) ;
- on considère les deux polygones suivants : l'un formé par les sommets $p_i, \dots, p_j$ et l'autre formé par les sommets $p_j, \dots, p_n, p_1, \dots, p_i$;
- (récurance) l'aire de P est égale à la somme des aires des deux polygones considérés au point précédent.

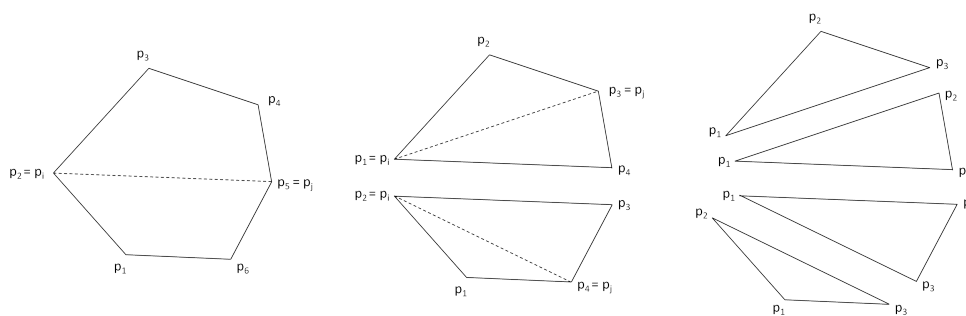


FIGURE 1 – Exemple d'étapes de récurrence pour un polygone à 6 sommets.

Question :

- a) Donnez le code python d'une fonction **réursive** `aire(liste)` qui étant donné une liste `liste` de tuples (couples), calcule l'aire du polygone formé par les couples contenus dans `liste`, en utilisant la méthode décrite précédemment.

Préconditions :

- `liste` contient des couples (tuples à deux composantes) ;
- `liste` représente les sommets d'un polygone convexe et ceux-ci sont énumérés dans le sens des aiguilles d'une montre.

Problème 3 : Un petit réseau social

On souhaite implémenter un petit réseau social en utilisant deux classes pour modéliser les personnes et le réseau.

La classe `Personne`

Pour cela, on modélise chaque personne par un objet `Personne` possédant trois attributs :

- `nom` : une chaîne de caractères contenant le nom de la personne
- `email` : une chaîne de caractères contenant son adresse email
- `amis` : une liste Python (`list`) d'objets de type `Personne`, représentant ses amis

La classe `Reseau`

Un réseau social est simplement une liste de personnes. On le modélise par un objet `Reseau` possédant un unique attribut :

- `personnes` : une liste Python (`list`) d'objets de type `Personne`

Questions :

- Ecrivez la méthode `non_narcissique(self)` de la classe `Reseau`. Cette méthode renvoie `True` si, parmi toutes les personnes du réseau, aucune ne fait partie de sa propre liste d'amis, et `False` sinon.
- Ecrivez la méthode `ajouter_un_ami(self, ami)` de la classe `Personne`. Cette méthode ajoute l'objet `ami` de type `Personne` dans la liste d'amis de la personne, uniquement si :
 - `ami` n'est pas la personne en question (c'est-à-dire `self`) et
 - `ami` n'est pas déjà dans la liste d'amis de la personne en question.
- Ecrivez la méthode `ajouter_les_amis_d_amis(self)` de la classe `Personne`. Cette méthode ajoute à la liste d'amis de la personne (`self`), tous les amis de ses amis, en respectant le fait que :
 - on n'ajoute pas la personne elle-même (c'est-à-dire `self`) et
 - on n'ajoute pas deux fois le même ami dans la liste d'amis.

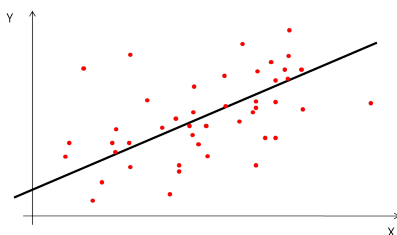
Remarques :

- Vous pouvez définir et utiliser des méthodes annexes aux classes `Personne` et `Reseau` si nécessaire.

Problème 4 : régression linéaire

Grossièrement, la régression linéaire est une méthode statistique permettant d'approximer au mieux un nuage de points par une droite. La meilleure droite, selon la méthode des moindres carrés, est celle qui minimise la somme des carrés des distances des points à la droite.

Dans le cadre de cet exercice, nous travaillerons dans le plan $\mathbb{R}^2$. Une droite aura alors pour équation $y = ax + b$ (droites verticales non traitées). Le but sera, à partir d'un ensemble de points, de déterminer les valeurs de a et b de la "meilleure droite" approximant ces points.



Soit $n \geq 2$. Soient $(x_1, y_1), \dots, (x_n, y_n)$ des points de $\mathbb{R}^2$. Voici quelques définitions qui vont vous permettre de calculer a et b .

Description	Notation	Valeur
moyenne des x_i	$\bar{x}$	$\frac{1}{n} \sum_{i=1}^n x_i$
moyenne des y_i	$\bar{y}$	$\frac{1}{n} \sum_{i=1}^n y_i$
variance des x_i	S_X^2	$\frac{1}{n} \sum_{i=1}^n (x_i - \bar{x})^2 = \overline{x^2} - \bar{x}^2$
covariance des x_i, y_i	S_{XY}	$\frac{1}{n} \sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y}) = \overline{x \cdot y} - \bar{x} \cdot \bar{y}$

A partir de ces informations, a et b sont obtenus comme suit :

$$a = \frac{S_{XY}}{S_X^2} \quad \text{et} \quad b = \bar{y} - \frac{\bar{x} \cdot S_{XY}}{S_X^2} = \bar{y} - a \cdot \bar{x}$$

Questions

- Donnez le code python d'une fonction `reg_lin(liste)` prenant en paramètre une liste de couples (tuples) de flottants (représentant les points (x_i, y_i)) et qui retourne un tuple de flottants (a, b) tel que a et b sont calculés grâce aux formules définies ci-dessus à partir des points de `liste`. Veuillez donner une brève description de votre code, en français. Vous devez faire en sorte que votre algorithme parcourt un minimum de fois la liste de couples. Vous ne devez pas vérifier que la liste entrée en paramètre est bien une liste de couples de flottants (précondition).
- Evaluez la complexité de votre algorithme (notation grand $\mathcal{O}$) en fonction de la taille de la liste entrée en paramètre. Justifiez.